

CRUSTPairs: A Cross-Language Dataset of Paired C and Rust Solutions

Leonardo J. Almeida
University of São Paulo
leonardo.j.almeida@usp.br

Vojtěch Hejlek
TODO affiliation
TODO email

Abstract—C-to-Rust migration is an active agenda, but no dataset pairs the two languages at the level of whole solutions to a shared problem. We present CRUSTPairs, 1857 C/Rust implementation pairs drawn from competitive programming archives, normalised, deduplicated, and stratified into difficulty tiers by zero-shot cross-language embedding similarity, with per-record provenance and licensing. Cross-language retrieval is hard on this data, and harder than a single baseline would suggest: TF-IDF over shared identifiers reaches MRR 0.21 and so beats a pretrained code model at 0.19, which on the hardest quarter of the data scores $1.7\times$ chance. Finetuning reaches 0.79, and lifts that hardest quarter $10\times$. Renaming every user identifier settles what is being learned: the finetuned models keep 97% of their accuracy without them, while the pretrained model keeps 66% — less than the lexical baseline does — and on structurally divergent pairs the finetuned models are *better* without identifiers than with them.

Index Terms—dataset, cross-language, code similarity, Rust, C, code retrieval

I. INTRODUCTION

Memory-safety defects remain the dominant vulnerability class in C. Microsoft has for two decades attributed roughly 70% of its CVEs to that class [1], and Android shows the substitution works: its annual memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024 as Rust adoption grew, with roughly $1000\times$ lower defect density in Rust than in C and C++ [2]. Policy has followed, through CISA’s memory-safe roadmaps [3] and the US cyber director’s call to adopt memory-safe languages [4]; DARPA’s TRACTOR programme [5] funds the automated translation of legacy C to Rust outright, on the premise that the codebases that matter are too large to port by hand.

Translation tools are consequently proliferating, and they are evaluated almost entirely on whether their output compiles and passes the original test suite [6], [7]. That is the right primary criterion. It is also incomplete — a translation can pass every test and still be an unfaithful rendering of its source — and it is unavailable precisely where it is most needed, since legacy C is what most often lacks tests. The complementary question, *are these two programs, in two languages, the same program?*, is a similarity judgement rather than an execution one, and answering it with a learned model requires paired supervision.

No such corpus exists for this language pair. The parallel code datasets either span many languages at snippet granularity [8], [9] or go deep on a single pair that is not this one [10], and the largest collections are grouped by problem rather than

aligned across languages [11], [12]. None of them includes Rust.

We present CRUSTPairs, 1857 pairs of C and Rust implementations of the same competitive programming problem, each with its statement, normalised entry points, per-record provenance and licence, a difficulty tier derived from zero-shot cross-language embedding similarity, and a frozen stratified split.

Measuring on it reframes what the task requires. A lexical baseline—TF-IDF over shared identifiers, no model—beats a pretrained code model zero-shot, because competitive programmers reuse variable names across languages. Renaming every user identifier separates the regimes: the pretrained model loses a third of its accuracy, more than the lexical baseline does, while finetuned models lose almost nothing. Cross-language matching here is learnable, but not from identifiers.

Contributions.

- CRUSTPairs, 1857 C/Rust solution pairs with problem statements, normalised entry points, per-record provenance and licensing.
- A difficulty stratification derived from zero-shot cross-language embedding similarity, independent of the model family used in evaluation, which we show transfers to a different family.
- Retrieval baselines per difficulty tier over a frozen stratified split, including a lexical baseline and an identifier ablation that together establish what the task does and does not reduce to.

II. RELATED DATASETS

Parallel code corpora divide by what they align and how many languages they span (Table I). The largest are not parallel at all: CodeNet [11] and POJ-104, distributed with CodeXGLUE [12], group solutions by problem without shipping an aligned bilingual corpus. Of the parallel ones, xCodeEval [13] is broadest at 17 languages; XLCOST [8] and CoST [9] align snippets rather than whole programs across the 7 languages GeeksForGeeks covers; and AVATAR [10] goes deep on Java/Python instead. None includes Rust, which is the gap CRUSTPairs fills and the reason it is one language pair rather than a matrix. We present CRUSTPairs, 1857 C/Rust implementation pairs drawn from competitive programming archives, normalised, deduplicated, and stratified into difficulty

tiers by zero-shot cross-language embedding similarity, with per-record provenance and licensing. Cross-language retrieval is hard on this data, and harder than a single baseline would suggest: TF-IDF over shared identifiers reaches MRR 0.21 and so beats a pretrained code model at 0.19, which on the hardest quarter of the data scores $1.7\times$ chance. Finetuning reaches 0.79, and lifts that hardest quarter $10\times$. Renaming every user identifier settles what is being learned: the finetuned models keep 97% of their accuracy without them, while the pretrained model keeps 66% — less than the lexical baseline does — and on structurally divergent pairs the finetuned models are *better* without identifiers than with them.

Index Terms—dataset, cross-language, code similarity, Rust, C, code retrieval

III. INTRODUCTION

Memory-safety defects remain the dominant vulnerability class in C. Microsoft has for two decades attributed roughly 70% of its CVEs to that class [1], and Android shows the substitution works: its annual memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024 as Rust adoption grew, with roughly $1000\times$ lower defect density in Rust than in C and C++ [2]. Policy has followed, through CISA’s memory-safe roadmaps [3] and the US cyber director’s call to adopt memory-safe languages [4]; DARPA’s TRACTOR programme [5] funds the automated translation of legacy C to Rust outright, on the premise that the codebases that matter are too large to port by hand.

Translation tools are consequently proliferating, and they are evaluated almost entirely on whether their output compiles and passes the original test suite [6], [7]. That is the right primary criterion. It is also incomplete — a translation can pass every test and still be an unfaithful rendering of its source — and it is unavailable precisely where it is most needed, since legacy C is what most often lacks tests. The complementary question, *are these two programs, in two languages, the same program?*, is a similarity judgement rather than an execution one, and answering it with a learned model requires paired supervision.

No such corpus exists for this language pair. The parallel code datasets either span many languages at snippet granularity [8], [9] or go deep on a single pair that is not this one [10], and the largest collections are grouped by problem rather than aligned across languages [11], [12]. None of them includes Rust.

We present CRUSTPairs, 1857 pairs of C and Rust implementations of the same competitive programming problem, each with its statement, normalised entry points, per-record provenance and licence, a difficulty tier derived from zero-shot cross-language embedding similarity, and a frozen stratified split.

Measuring on it produces a result that reframes what the task requires. A lexical baseline — TF-IDF over shared identifiers, no model — beats a pretrained code model zero-shot, because competitive programmers reuse variable names across languages. Renaming every user identifier separates the two regimes cleanly: the pretrained model loses a third of its

accuracy, more than the lexical baseline loses, while finetuned models lose almost nothing and on structurally divergent pairs do *better* without names than with them. Cross-language matching here is learnable, but not from identifiers, and the dataset is what makes that distinction measurable.

Contributions.

- CRUSTPairs, 1857 C/Rust solution pairs with problem statements, normalised entry points, per-record provenance and licensing.
- A difficulty stratification derived from zero-shot cross-language embedding similarity, independent of the model family used in evaluation, which we show transfers to a different family.
- Retrieval baselines per difficulty tier over a frozen stratified split, including a lexical baseline and an identifier ablation that together establish what the task does and does not reduce to.

IV. RELATED DATASETS

Parallel code corpora divide by what they align and how many languages they span, and Table I places CRUSTPairs among them.

The largest are not parallel at all. CodeNet [11] collects roughly 14 million submissions across 55 languages from AtCoder and AIZU, and POJ-104, distributed as part of CodeXGLUE [12], holds student C++ submissions to 104 problems. Both group solutions by problem, so pairs can be derived, but neither ships an aligned bilingual corpus and both are dominated by C, C++, Java and Python.

Among the explicitly parallel corpora, xCodeEval [13] is the broadest, with 25 million Codeforces examples over 17 languages and execution-based evaluation through unit tests. XLCoST [8] and CoST [9] align at the finer granularity of code snippets rather than whole programs, across 7 languages drawn from GeeksForGeeks; the alignment is what makes them valuable and also what restricts them to the languages that site covers. AVATAR [10] takes the opposite tack and goes deep on one pair, 9,515 Java/Python problems with unit tests for a 250-example subset.

None of them includes Rust. That is the gap CRUSTPairs fills, and the reason it is a single language pair rather than a broad matrix: the pair that matters for the migration pressure described in Section VIII is C to Rust specifically, and no existing corpus aligns those two at the level of whole solutions to a shared problem.

The modelling literature this dataset is meant to serve is correspondingly single-pair-poor. Unsupervised program translation was established on C++/Java/Python by TransCoder [14], and the pretrained code encoders that cross-language retrieval is built on—CodeBERT [15] and UniXcoder [16]—were pretrained on corpora in which Rust is marginal or absent. Cross-language clone detection has the same shape: CLCDSA [17] detects type-4 clones across Java, C# and Python using syntactic features and API-call similarity, and its evaluation, like the corpora above, stops short of Rust.

TABLE I
CROSS-LANGUAGE CODE CORPORA. CRUSTPAIRS IS THE ONLY ONE
PAIRING C WITH RUST AT SOLUTION GRANULARITY. TODO: VERIFY
EVERY CELL.

Dataset	Languages	Paired	Rust
CodeNet [11]	55	no	no
xCodeEval [13]	17	yes	no
XLCoST [8]	7	yes	no
CoST [9]	7	yes	no
AVATAR [10]	2	yes	no
POJ-104 [12]	1	n/a	no
CRUSTPairs	2	yes	yes

On the C-to-Rust side specifically, the recent work is on *producing* Rust rather than on judging whether the result corresponds to its source. C2SaferRust [6] decomposes C2Rust’s unsafe output into slices an LLM rewrites more safely, and Syzygy [7] pairs LLM translation with dynamic analysis to keep code and tests consistent. Both are evaluated by compilation and test-passing, which is the criterion Section VIII argues is necessary and not sufficient.

V. CONSTRUCTION

A. Sources and pairing

CRUSTPairs draws on two competitive programming archives: xCodeEval [13], itself derived from Codeforces, and IBM Project CodeNet [11], derived from AtCoder. Both distribute accepted submissions, so the two implementations of a pair are known to have passed the same official tests. A pair is one problem, one accepted C submission, and one accepted Rust submission.

B. Normalisation and cleaning

Both sides pass through a static analysis pipeline that strips comments, removes unused functions and macros, normalises encoding, and applies the language’s standard formatter. Entry points are renamed to `solution` in both languages so the function name carries no signal—which also means the distributed code no longer compiles standalone (Section IX).

From an initial 2013 pairs, 127 were removed: 2 confirmed mis-pairs, 69 AtCoder easy/hard variants with near-identical statements (description embedding similarity > 0.90), 54 further suspicious pairs (0.80–0.90) found the same way with OpenAI `text-embedding-3-large`, 1 length-ratio outlier and 1 case of genuine implementation divergence. A further 29 textbook reference implementations were dropped: they carried the only copyleft content and the only records whose halves were under different licences (Section X), and had no problem statement. This leaves 1857 pairs.

C. Difficulty stratification

Each pair is scored by the cosine similarity between zero-shot SFR-Embedding-Code-400M_R embeddings of its C and Rust sides, and binned at the 25th and 75th percentiles into easy, medium and hard (thresholds 0.7856 and 0.8687; similarity mean 0.826, standard deviation 0.058). The raw

TABLE II
SCHEMA. ONE JSON OBJECT PER LINE OF `DATASET.JSONL`; ALL FIELDS
ARE STRINGS EXCEPT `DIFFICULTY_SCORE`.

Field	Values
<code>problem_id</code>	0001–1886, inherited
<code>origin</code>	codenet (839), xcodeeval (1018)
<code>license</code>	SPDX expression; follows origin
<code>problem_description</code>	upstream statement, verbatim
<code>..._format</code>	html (839), text (1018)
<code>c_code</code>	entry point renamed <code>solution</code>
<code>rust_code</code>	entry point renamed <code>solution</code>
<code>difficulty</code>	easy (465), medium (928), hard (464)
<code>difficulty_score</code>	raw SFR cosine similarity (float)
<code>split</code>	train (1485), val (185), test (187)

score is distributed alongside the tier, so the cuts can be redrawn without re-running the model. Pairs a generic code model already embeds close together are near-literal translations; pairs it separates diverge structurally and demand a real cross-language remapping.

The choice of SFR rather than UniXcoder [16] is deliberate. Defining the tiers with the same model family that is later finetuned would make hard pairs hard by construction, and any reported per-tier improvement would be circular. SFR is trained on a different corpus, so the tiers remain an independent axis. The stratification is also origin-agnostic: AtCoder and Codeforces publish difficulty ratings on incompatible scales, so upstream ratings are not used.

VI. DATASET DESCRIPTION

A. Schema

CRUSTPairs is a single 5.9 MB JSON Lines file, one object per pair, with the fields in Table II. Nothing is split across files and no field needs to be joined from elsewhere.

The fifth row is `problem_description_format`, abbreviated to `fit`; `problem_id` runs to 1886 over 1857 records, keeping an earlier release’s numbering. Two choices are worth naming. `license` is per record, because the sources differ (Section X), so filtering on `origin` is the only way to extract a commercially usable subset. And `difficulty_score` ships the raw similarity beside the tier, so the quantile cuts can be redrawn without re-running the model.

B. Statistics

Solutions are short: a median of 516 characters of C and 749 of Rust, with problem statements at a median of 1231. The Rust side runs about $1.5\times$ longer in characters (median ratio 1.50) while being *shorter* in lines—31.4 against 33.6 on average—so Rust solutions pack more into each line. A length-based heuristic therefore does not separate the two languages the way the character counts alone would suggest.

Source and difficulty are not independent. CodeNet contributes 241 easy, 408 medium and 190 hard pairs; xCodeEval 224, 520 and 274. The CodeNet portion skews easy while

TABLE III
CROSS-LANGUAGE RETRIEVAL ON THE 187-PAIR TEST SET.

Model	MRR	MRR@10	R@1	R@5	R@10
Chance	0.031	0.016	0.005	0.027	0.053
Lexical TF-IDF	0.206	0.191	0.128	0.273	0.353
UniXcoder, zero-shot	0.189	0.170	0.134	0.203	0.283
UniXcoder + LoRA	0.724	0.720	0.624	0.836	0.913
UniXcoder, full	0.792	0.791	0.706	0.909	0.952

the xCodeEval portion skews hard. Reporting a per-tier result therefore requires a split that controls for both.

C. Splits

The train/validation/test assignment is frozen in the data rather than regenerated from a seed, and is stratified by source $\times$ difficulty at 80/10/10: 1485, 185 and 187 pairs. The test set mirrors the full dataset to within 0.3 percentage points by source and 0.6 by tier.

VII. BASELINES

Retrieval is C-to-Rust unless stated otherwise: each C implementation queries the pool of all 187 Rust implementations in the test split, and the target is the one solving the same problem.

We report mean reciprocal rank, introduced for the TREC-8 question answering track [18] and the standard measure for code search since CodeSearchNet [19] and CodeXGLUE [12]: each query contributes the reciprocal of the rank at which its correct counterpart appears, so the score is dominated by whether the answer lands at the top rather than merely somewhere in the list. MRR@10 is the same quantity truncated, contributing zero when the rank exceeds ten. Both are given because they diverge exactly where many positives fall outside the top ten, which is the zero-shot regime and therefore the denominator of every improvement ratio below. Chance at this pool size is $\text{MRR} = 0.031$. LoRA figures are mean $\pm$ standard deviation over four seeds; the other rows are single runs.

The lexical row is the one that fixes the frame. TF-IDF over shared identifiers, with no model at all, reaches 0.206 and so *beats* zero-shot UniXcoder [16] at 0.189. Competitive programmers reuse variable names across languages: *n* is shared by 85 of the 187 test pairs, *i* by 84, then *a*, *t*, *s*, *j*. The claim this dataset supports is therefore not that a generic code model finds the task hard, but the stronger one that *neither* generic code embeddings *nor* lexical matching solve it, and finetuning does — $3.8\times$ over zero-shot for LoRA [20] and $4.2\times$ for full finetuning ($4.2\times$ and $4.7\times$ on MRR@10, which diverges from MRR because many zero-shot positives fall outside the top ten).

A. Difficulty tiers and the identifier ablation

Table IV reports MRR per tier, and repeats every configuration with user identifiers renamed to positional names independently in each language, which destroys cross-language

TABLE IV
MRR BY DIFFICULTY TIER, WITH AND WITHOUT USER IDENTIFIERS. EVERY TIER IS SCORED AGAINST THE SAME 187-CANDIDATE POOL; ONLY THE QUERIES ARE PARTITIONED. *Ret.* IS THE ANONYMISED MRR AS A FRACTION OF THE NORMAL ONE.

Model	Ident.	Easy	Medium	Hard	Ret.
Lexical TF-IDF	kept anon	0.446 0.355	0.144 0.094	0.089 0.080	76%
UniXcoder, zero-shot	kept anon	0.328 0.276	0.186 0.084	0.053 0.048	66%
UniXcoder + LoRA	kept anon	0.957 0.941	0.776 0.688	0.388 0.429	96%
UniXcoder, full	kept anon	0.979 0.951	0.821 0.756	0.550 0.611	97%

name correspondence while leaving control flow, types, library calls and string literals intact.

Three things follow, and together they are the strongest evidence that this dataset measures what it is meant to.

Retrieval is monotone in tier for every configuration. The tiers were cut with SFR while the models here are UniXcoder-based, so the ordering is a property of the pairs rather than an artefact of the encoder that defined it. Zero-shot UniXcoder reaches 0.053 on the hard tier — $1.7\times$ chance, with exactly one of the 47 hard pairs at rank 1. A generic code model effectively cannot solve a quarter of this dataset.

Finetuning learns structure, not names. The finetuned models keep 96–97% of their MRR when identifiers are destroyed; zero-shot UniXcoder keeps 66%, which is *less* than the 76% the lexical baseline keeps. A pretrained code model is more dependent on variable names than TF-IDF is. That inverts the natural reading of Table III: zero-shot UniXcoder does not merely fail to beat lexical matching, it is doing a worse version of the same thing.

On hard pairs, identifiers are a liability. Both finetuned models do *better* without them — LoRA from 0.388 to 0.429, full finetuning from 0.550 to 0.611, both about +11% — while the lexical baseline and the zero-shot model get worse. Where implementations diverge structurally, a shared name is more often a coincidence than a signal, and a model trained without that cue learns the remapping instead of leaning on it.

B. Direction and parameter efficiency

Querying with Rust beats querying with C (MRR 0.189 to 0.233 zero-shot, 0.792 to 0.836 full), and the asymmetry nearly vanishes under the ablation (0.768 against 0.773), locating it in identifiers rather than structure. LoRA reaches 91% of full finetuning overall with 0.23% of the parameters, but 98% on easy pairs against 71% on hard: the budget binds where the remapping is non-trivial, and the aggregate ratio conceals it.

VIII. USE CASES

Section III set out why C-to-Rust translation is being funded rather than merely discussed. What follows is the gap between that effort and what it currently measures.

A. What a paired corpus contributes

Translation tools are evaluated almost entirely on whether their output compiles and passes the original tests. TRAC-TOR’s own benchmark is a battery of 150 C programs scored for correctness and safety [21]. That is the right primary criterion and an incomplete one: a translation can pass every test and still be an unfaithful or unidiomatic rendering of the source, and a test suite is exactly what legacy C most often lacks. The complementary question — *are these two programs, in two languages, the same program?* — is a similarity judgement, and it is the one CRUSTPairs supplies supervision for.

Four concrete uses follow.

Scoring translator output without tests. A model fine-tuned on CRUSTPairs assigns a cross-language similarity to a (C, Rust) pair. Applied to a translator’s input and output it gives a test-free signal that flags translations drifting from their source — usable as a regression check across tool versions, or to rank candidates when a tool emits several. The per-tier results in Section VII matter here: the usable range of such a score is set by how well a model does on *structurally divergent* pairs, which is where the hard tier shows the current ceiling is 0.55 MRR, not the 0.98 the easy tier would suggest.

Finding an existing port. Migrating a C codebase rarely starts from nothing: equivalent functionality often already exists in Rust, in a crate or in a sibling project. Locating it is cross-language retrieval over whole implementations, which is the task this dataset measures directly.

Cross-language clone and vulnerability tracking. When a CVE is filed against a C implementation, the question of whether a Rust port inherits the defect — or whether a Rust crate is a translation of vulnerable C — is a type-4 clone problem across languages. Type-4 clone corpora are overwhelmingly single-language; a paired C/Rust corpus gives the cross-language case supervision it has lacked.

B. What it does not support

The pairs are competitive programming solutions: short, single-file, I/O-driven, and free of the allocation patterns, pointer aliasing and concurrency that make real C hard to port. Results here should be read as a measurement on a clean proxy for the cross-language matching problem, not as evidence about systems code, which is where the migration pressure described above actually sits. Section IX returns to this.

IX. LIMITATIONS

Equivalence is inherited, not verified. No tests or expected I/O are distributed. That both submissions were accepted upstream is evidence of functional equivalence, not a check of it, and the entry-point renaming means the code does not compile standalone. Execution-based verification is the clearest line of future work.

Competitive programming is not production code. Solutions are short, single-file and I/O-driven; nothing here should

be assumed to transfer to systems code, which is where the migration pressure actually sits.

Upstream identifiers are absent, so pairs cannot be traced to the originating submission, and **difficulty is a proxy** for embedding similarity rather than algorithmic difficulty or translation effort.

Variance is measured for one configuration. The LoRA rows are means over four seeds ($\sigma = 0.011$ on MRR); the others are single runs, so that spread is the only direct evidence of how much of any gap is seed noise.

An earlier release’s tier labels were not exactly reproducible. Regenerating them matches the aggregate distribution (mean 0.826 against a reported 0.823) but only 89% of individual labels, with disagreement spread symmetrically across adjacent tiers—a pattern threshold movement alone cannot produce. The labels released here are the regenerated ones, and the raw scores now ship alongside them. **Pre-training contamination is also possible,** both archives being public.

X. AVAILABILITY AND LICENSING

CRUSTPairs aggregates two corpora under different licences, so it is not distributed under a single one; every record carries its own SPDX expression. The 1018 xCodeEval-derived pairs are CC BY-NC 4.0 and the 839 CodeNet-derived pairs are CDLA-Permissive-2.0. The aggregate is therefore non-commercial, and attribution to both upstream projects is required. Filtering to the CodeNet portion yields a 839-pair subset with no commercial restriction.

ACKNOWLEDGEMENTS

TODO.

REFERENCES

- [1] M. Russinovich, “Keynote,” RustConf, 2025, reports that roughly 70% of Windows CVEs are memory-safety issues.
- [2] Google, “Rust in Android: Move fast and fix things,” Google Security Blog, 2025, android memory-safety vulnerabilities fell from 223 in 2019 to under 50 in 2024, below 20% of the total.
- [3] Cybersecurity and Infrastructure Security Agency, “The case for memory safe roadmaps,” Dec. 2023.
- [4] Office of the National Cyber Director, “Back to the building blocks: A path toward secure and measurable software,” The White House, Feb. 2024.
- [5] Defense Advanced Research Projects Agency, “Eliminating memory safety vulnerabilities once and for all,” DARPA news release, TRAC-TOR programme (Translating All C to Rust), Jul. 2024. [Online]. Available: <https://www.darpa.mil/news/2024/memory-safety-vulnerabilities>
- [6] V. Nitin, R. Krishna, L. Lemos do Valle, and B. Ray, “C2SaferRust: Transforming C projects into safer Rust with neurosymbolic techniques,” arXiv preprint arXiv:2501.14257, 2025.
- [7] M. Shetty, N. Jain, A. Godbole, S. A. Seshia, and K. Sen, “Syzygy: Dual code-test C to (safe) Rust translation using LLMs and dynamic analysis,” in *Proceedings of the 2nd International Workshop on Large Language Models for Code (LLM4Code)*, 2025.
- [8] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, “XLCOST: A benchmark dataset for cross-lingual code intelligence,” 2022.
- [9] M. Zhu, K. Suresh, and C. K. Reddy, “Multilingual code snippets training for program translation,” in *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, 2022.
- [10] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, “AVATAR: A parallel corpus for Java-Python program translation,” in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.

- [11] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker, V. Thost, L. Buratti, S. Pujar, S. Ramji, U. Finkler, S. Malaika, and F. Reiss, "CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [12] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [13] M. A. M. Khan, M. S. Bari, D. X. Long, W. Wang, M. R. Parvez, and S. Joty, "xCodeEval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," arXiv preprint, 2023.
- [14] B. Roziere, M.-A. Lachaux, L. Chausson, and G. Lample, "Unsupervised translation of programming languages," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [15] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [16] D. Guo, S. Lu, N. Duan, J. Yin, D. Jiang, and M. Zhou, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [17] K. W. Nafi, T. S. Kar, B. Roy, C. K. Roy, and K. A. Schneider, "CLCDSA: Cross language code clone detection using syntactical features and API documentation," in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019.
- [18] E. M. Voorhees, "The TREC-8 question answering track report," in *Proceedings of the 8th Text REtrieval Conference (TREC-8)*, 1999, introduces mean reciprocal rank as the track's primary measure.
- [19] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," arXiv preprint arXiv:1909.09436, 2019.
- [20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.
- [21] MIT Lincoln Laboratory, "Translating all C to Rust (TRACTOR) benchmarks," Project page. [Online]. Available: <https://www.ll.mit.edu/r-d/projects/translating-all-c-rust-tractor-benchmarks>